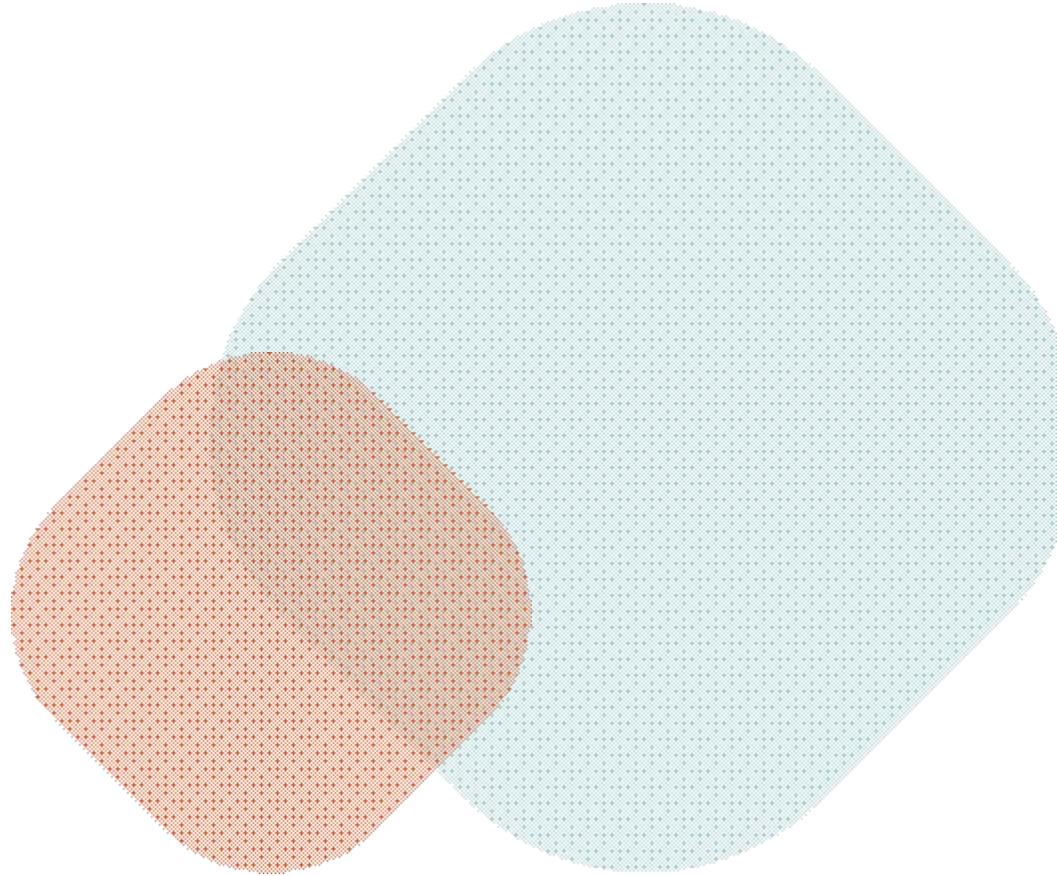
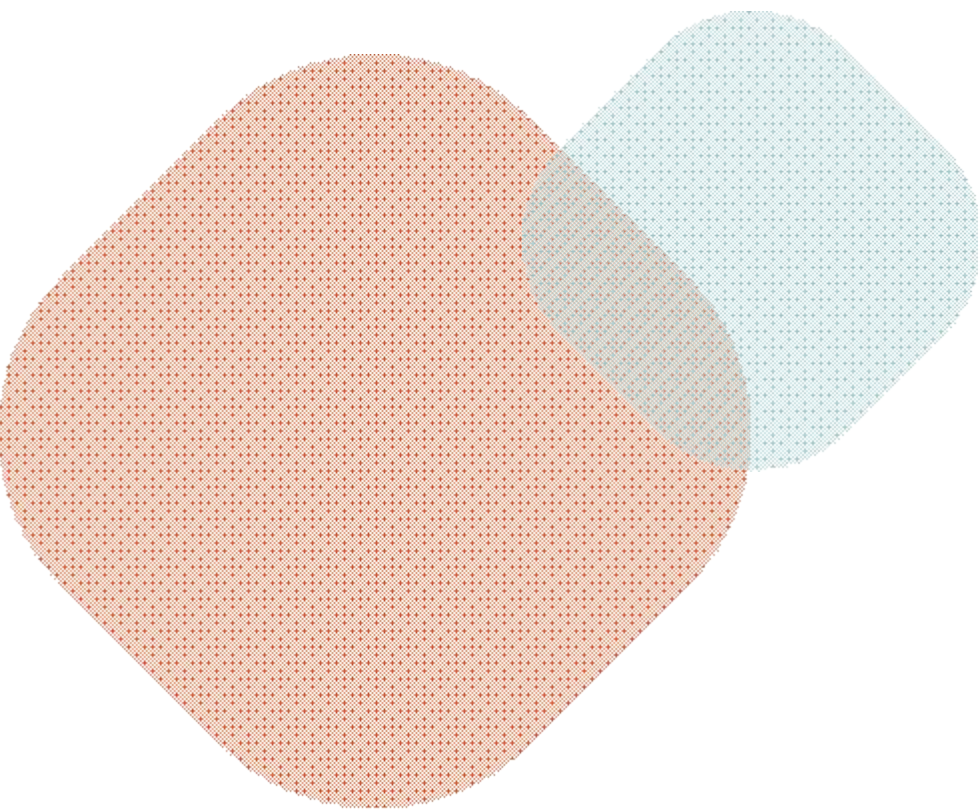




축구 데이터를 이용한 승부 예측 모델

2019380505 서영빈
2019380703 임찬희

PART
01

“
데이터 수집
”

주제 선정 배경

데이터 분석의 필요성

축구 경기는 변수가 많아 예측이 어려움



빅데이터와 머신러닝 기술 발전



경기 결과 예측 분야에서도 활용되고 있음

실용적인 측면

축구 경기에 대한 데이터 분석은
축구 관련 산업 뿐 아니라
경영, 금융 등 다양한 분야에서 활용 가능함



데이터 수집

Kaggle에 있는 데이터 활용

Season	DateTime	HomeTeam	AwayTeam	FTHG	FTAG	FTR	HTHG	HTAG	HTR	Referee	HS	AS	HST	AST	HC	AC	HF	AF	HY	AY
1993-94	1993-08-	Arsenal	Coventry	0		3 A	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
1993-94	1993-08-	Aston Villa	QPR	4		1 H	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
1993-94	1993-08-	Chelsea	Blackburn	1		2 A	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
1993-94	1993-08-	Liverpool	Sheffield W	2		0 H	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
2021-22	2022-04-	Watford	Leeds	0		3 A		0	1 A	A Mariner	8	10	1		5	3	8	8	12	2
2021-22	2022-04-	Aston Villa	Tottenham	0		4 A		0	1 A	G Scott	9	11	8		5	9	3	12	14	2
2021-22	2022-04-	Brentford	West Ham	2		0 H		0	0 D	M Atkinson	15	5	7		1	4	6	2	6	0
2021-22	2022-04-	Leicester	Crystal Pal	2		1 H		2	0 H	R Jones	12	11	3		3	3	4	11	12	1
2021-22	2022-04-	Norwich	Burnley	2		0 H		1	0 H	M Oliver	17	18	6		4	6	7	12	10	1
2021-22	2022-04-	Man City	Liverpool	2		2 D		2	1 H	A Taylor	11	6	5		4	4	1	9	11	1

총 22개의 변수와 11114개의 자료 수집



PART
02

“
데이터 전처리
”

#	SEASON	DATE	TIME	HOME_TEAM	AWAY_TEAM	HOME_GOALS	AWAY_GOALS
...	...	...	...	...	...	...	...
11108	2021-22	2022-04-09	T17:30:00Z	Aston Villa	Tottenham	0	4
11109	2021-22	2022-04-10	T14:00:00Z	Brentford	West Ham	2	0
11110	2021-22	2022-04-10	T14:00:00Z	Leicester	Crystal Palace	2	1
11111	2021-22	2022-04-10	T14:00:00Z	Norwich	Burnley	2	0
11112	2021-22	2022-04-10	T16:30:00Z	Man City	Liverpool	2	2

	FTR	HTHG	HTAG	HTR	...	HST	AST	HC	AC	HF	AF	HY	AY	#
0	A	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
1	H	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2	A	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
3	H	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
4	D	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

경기 초반의 결과가 오래 되어서 정보가 없는 문제 발생


```
In [11]: df = df.iloc[2825:]
print(df.head())
```

	Season	DateTime	HomeTeam	AwayTeam	FTHG	FTAG	FTR	Ⓜ
2870	2000-01	2000-09-09T00:00:00Z	Newcastle	Chelsea	0	0	D	
2871	2000-01	2000-09-10T00:00:00Z	Derby	Charlton	2	2	D	
2872	2000-01	2000-09-11T00:00:00Z	Tottenham	West Ham	1	0	H	
2873	2000-01	2000-09-16T00:00:00Z	Arsenal	Coventry	2	1	H	
2874	2000-01	2000-09-16T00:00:00Z	Aston Villa	Bradford	2	0	H	

	HTHG	HTAG	HTR	...	HST	AST	HC	AC	HF	AF	HY	AY	HR	Ⓜ
2870	0.0	0.0	D	...	4.0	3.0	5.0	6.0	17.0	21.0	0.0	3.0	0.0	
2871	2.0	0.0	H	...	6.0	6.0	4.0	11.0	17.0	12.0	3.0	1.0	0.0	
2872	0.0	0.0	D	...	7.0	5.0	11.0	3.0	14.0	13.0	1.0	2.0	0.0	
2873	1.0	0.0	H	...	13.0	2.0	8.0	4.0	5.0	13.0	1.0	2.0	0.0	
2874	1.0	0.0	H	...	7.0	4.0	5.0	6.0	8.0	12.0	0.0	2.0	0.0	

	AR
2870	0.0
2871	0.0
2872	0.0
2873	0.0

결측값을 대체하는 방법을 찾는 와중에 대체 후 문제점이 발생해
정보가 없는 2800개의 데이터를 제외하고 9000개의 데이터를 활용해 분석

독립변수 재설정

```
# 독립변수와 종속변수 선택
X = df[['HTR', 'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF', 'AF', 'HY', 'AY', 'HR', 'AR']]
y = df['FTR']
```

```
FTHG 이상치: 265      6
336      7
754      7
787      9
1019     6
..
10631     9
10885     7
10963     7
10973     6
11050     6
Name: FTHG, Length: 95, dtype: int64
FTAG 이상치: 32      5
92      5
244     5
325     5
400     5
..
10874     5
10887     5
```



특점에 대한 이상치는
처리하지 않고
분석에 사용

PART
03

“
데이터 분석
”

#로지스틱 회귀분류

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

독립변수와 종속변수 선택

```
X = df[['HTR', 'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF', 'AF', 'HY', 'AY', 'HR', 'AR']]
y = df['FTR']
```

FTR을 범주형으로 변환

```
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)
```

데이터 분할

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

특성 스케일링

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

로지스틱 회귀분류 모델 생성 및 학습

```
logreg = LogisticRegression()
logreg.fit(X_train_scaled, y_train)
```

테스트 데이터로 예측

```
y_pred = logreg.predict(X_test_scaled)
```

모델 평가

```
accuracy = logreg.score(X_test_scaled, y_test)
print("정확도:", accuracy)
```

정확도: 0.6601106330669945

#관별분석

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

독립변수와 종속변수 선택

```
X = df[['HTR', 'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF', 'AF', 'HY', 'AY', 'HR', 'AR']]
y = df['FTR']
```

FTR을 범주형으로 변환

```
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)
```

데이터 분할

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

특성 스케일링

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

관별 분석 모델 생성 및 학습

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_train_scaled, y_train)
```

테스트 데이터로 예측

```
y_pred = lda.predict(X_test_scaled)
```

모델 평가

```
accuracy = lda.score(X_test_scaled, y_test)
print("정확도:", accuracy)
```

정확도: 0.6644130301167793

SVM

```
# Support Vector Machine (SVM)

from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

# 독립변수와 종속변수 선택
X = df[['HTR', 'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF', 'AF', 'HY', 'AY', 'HR', 'AR']]
y = df['FTR']

# FTR을 범주형으로 변환
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 데이터 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# SVM 모델 생성 및 학습
svm = SVC(kernel='linear', random_state=42)
svm.fit(X_train_scaled, y_train)

# 테스트 데이터로 예측
y_pred = svm.predict(X_test_scaled)

# 모델 평가
accuracy = svm.score(X_test_scaled, y_test)
print("정확도:", accuracy)
```

KNN, 로지스틱 회귀분석,
판별분석, SVM 등 분석들을
써본 결과 정확도가 낮음



정확도를 높이는 방법 모색

정확도: 0.6625691456668715

PART
04

“
데이터 추가
”

독립변수, 종속변수 설정

독립변수와 종속변수 선택

```
X = df[['home_clearances', 'home_corners', 'home_fouls_conceded',  
        'home_offsides', 'home_passes', 'home_possession', 'home_red_cards',  
        'home_shots_on_target', 'home_tackles', 'home_touches', 'home_yellow_cards',  
        'away_clearances', 'away_corners', 'away_fouls_conceded',  
        'away_offsides', 'away_passes', 'away_possession', 'away_red_cards',  
        'away_shots_on_target', 'away_tackles', 'away_touches', 'away_yellow_cards',  
        'clearances_avg_home', 'corners_avg_home', 'fouls_conceded_avg_home',  
        'offsides_avg_home', 'passes_avg_home', 'possession_avg_home',  
        'red_cards_avg_home', 'shots_on_target_avg_home', 'tackles_avg_home',  
        'touches_avg_home', 'yellow_cards_avg_home', 'clearances_avg_away',  
        'corners_avg_away', 'fouls_conceded_avg_away', 'offsides_avg_away',  
        'passes_avg_away', 'possession_avg_away', 'red_cards_avg_away',  
        'shots_on_target_avg_away', 'tackles_avg_away', 'touches_avg_away',  
        'yellow_cards_avg_away', 'year', 'month', 'day', 'goal_home_ht', 'goal_away_ht']]  
y = df['result']
```

기존에 있던 데이터의 변수가 부족하다고 생각하여
새로운 변수가 있는 데이터 추가 수집


```
# result를 범주형으로 변환
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 특성 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 로지스틱 회귀분류 모델 생성 및 학습
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_scaled, y_train)

# 테스트 데이터로 예측
y_pred = logreg.predict(X_test_scaled)

# 모델 평가
accuracy = logreg.score(X_test_scaled, y_test)
print("정확도:", accuracy)
```

독립변수 재설정 후
분석 재시도

정확도: 0.6801040312093628

```
# 판별 분석 모델 생성
lda = LinearDiscriminantAnalysis()

# 그리드 서치를 위한 매개변수 그리드 정의
param_grid = {'solver': ['svd', 'lsqr', 'eigen'],
              'shrinkage': [None, 'auto']}

# 그리드 서치 객체 생성 및 학습
grid_search = GridSearchCV(lda, param_grid, cv=5)
grid_search.fit(X_train_scaled, y_train)

# 최적 매개변수와 그에 대한 모델의 성능 출력
print("최적 매개변수:", grid_search.best_params_)
print("최고 정확도:", grid_search.best_score_)

# 테스트 데이터로 예측
y_pred = grid_search.predict(X_test_scaled)

# 모델 평가
accuracy = grid_search.score(X_test_scaled, y_test)
print("정확도:", accuracy)
```

```
최적 매개변수: {'shrinkage': None, 'solver': 'svd'}
최고 정확도: 0.6486174714509303
정확도: 0.6749095295536791
```



그리드 서치를 이용한
최적의 매개변수 탐색

	이전 데이터	정확도	그리드 서치
로지스틱 회귀분석	0.660	0.680	0.697
SVM	0.647	0.662	0.706
판별분석	0.644	0.668	0.674
랜덤포레스트	0.645	0.653	0.677

이전 자료들의 정확도에 비해
전체적으로 상승


```
# Bagging
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_sc = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500, max_samples=1000, bootstrap=True)
bag_sc.fit(X_train, y_train)
y_pred = bag_sc.predict(X_test)

from sklearn.metrics import accuracy_score
print(accuracy_score(y_test, y_pred))

0.7077641277641277
```

Decision Tree 개수 500
Sample 개수 500

이전 데이터 자료
64%



현재 데이터 자료
70%

```
# Pasting
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

# 의사결정나무 생성
base_estimator = DecisionTreeClassifier(max_depth=3)

# Pasting 앙상블 분류기 생성
pasting = BaggingClassifier(base_estimator=base_estimator, n_estimators=100, max_samples=1.0, bootstrap=False, random_state=1)

# 모델 학습
pasting.fit(X_train, y_train)

# 예측
y_pred = pasting.predict(X_test)

# 정확도 계산
accuracy = accuracy_score(y_test, y_pred)
print("정확도:", accuracy)
```

```
C:\Users\W82103\anaconda3\lib\site-packages\sklearn\ensemble\_base.py:166: FutureWarning: `base_estimator` was renamed to `estimator` in
version 1.2 and will be removed in 1.4.
  warnings.warn(
```

정확도: 0.6865110565110565

이전 데이터 자료
61%



현재 데이터 자료
68%

```
# adaboost
from sklearn.ensemble import AdaBoostClassifier
ada_sc = AdaBoostClassifier(DecisionTreeClassifier(max_depth=2), n_estimators=800, random_state=1)
ada_sc.fit(X_train, y_train)
y_train_pred=ada_sc.predict(X_train)
y_test_pred=ada_sc.predict(X_test)

from sklearn.metrics import accuracy_score
ada_train=accuracy_score(y_train, y_train_pred)
ada_test=accuracy_score(y_test, y_test_pred)

print("Adaboost train/test accuracy %0.3f/%0.3f" %(ada_train, ada_test))
```

Adaboost train/test accuracy 0.906/0.702

이전 데이터 자료
62%



현재 데이터 자료
70%


```
# 기물기 부스팅
from sklearn.ensemble import GradientBoostingClassifier
import numpy as np
gbcl = GradientBoostingClassifier(n_estimators=100, max_depth=2)
gbcl.fit(X_train, y_train)
accuracies=[accuracy_score(y_test,y_pred) for y_pred in gbcl.staged_predict(X_test)]
best_n_estimator=np.argmax(accuracies)
gbcl_best=GradientBoostingClassifier(max_depth=2, n_estimators=best_n_estimator)
gbcl_best.fit(X_train, y_train)
y_train_pred=gbcl_best.predict(X_train)
y_test_pred=gbcl_best.predict(X_test)
print(accuracy_score(y_train, y_train_pred))
print(accuracy_score(y_test, y_test_pred))
print(best_n_estimator) # 정밀도가 가장 높은 수
```

0.8152948402948403
0.7047911547911548
98

이전 데이터 자료
64%



현재 데이터 자료
70%

데이터를 학습 및 테스트 세트로 분할

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

XGBoost의 DMatrix 생성 시 enable_categorical 매개변수를 True로 설정

```
dtrain = xgb.DMatrix(X_train, label=y_train, enable_categorical=True)
```

```
dtest = xgb.DMatrix(X_test, label=y_test, enable_categorical=True)
```

XGBoost 모델 초기화

```
xg_clf = xgb.XGBRegressor(objective='multi:softmax', booster='gbtree', colsample_bytree=0.75,  
                           learning_rate=0.1, max_depth=3, alpha=10, n_estimators=50, num_class=3)
```

모델 학습

```
xg_clf.fit(X_train, y_train, eval_set=[(X_test, y_test)])
```

테스트 데이터에 대한 예측

```
y_pred = xg_clf.predict(X_test)
```

정확도 계산

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Accuracy: 0.7183715319662243

이전 데이터 자료
64%



현재 데이터 자료
71%

	이전 데이터	정확도
Bagging	0.644	0.707
Pasting	0.617	0.686
Adaboost	0.629	0.702
기울기부스팅	0.641	0.704
XGBoost	0.648	0.718

이전 자료들의 정확도에 비해
전체적으로 상승

PART
05

“
최종 모델 구축
”

영향을 미치는 변수 상위 10개

home_clearances
away_clearances
home_shots_on_target
away_shots_on_target
away_touches
away_passes
away_possession
home_possession
home_passes
touches_avg_away

홈 팀의 클리어런스 수
원정 팀의 클리어런스 수
홈 팀의 유효 슈팅 수
원정 팀의 유효 슈팅 수
원정 팀의 터치 수
원정 팀의 패스 수
원정 팀의 점유율
홈 팀의 점유율
홈 팀의 패스 수
원정 팀의 평균 터치 수

```
# 예측 함수
def predict_match_result(home_team, away_team):
    # 데이터프레임 생성
    data = pd.DataFrame(columns=X.columns)
    data.loc[0] = 0
    data['result'] = 0 # 'result' 열 추가 및 초기화

    # 홈팀과 어웨이팀 원-핫 인코딩
    home_team_encoded = 'home_team_' + home_team
    away_team_encoded = 'away_team_' + away_team

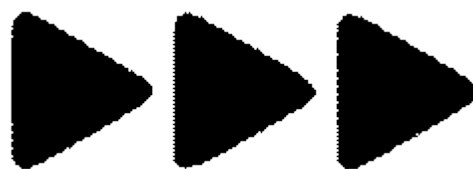
    # 예측에 필요한 특성만 선택 (결과 열은 제외)
    data = data.drop('result', axis=1)

    # 홈팀과 어웨이팀 원-핫 인코딩 추가
    data[home_team_encoded] = 1
    data[away_team_encoded] = 1

    # 훈련할 때와 예측할 때의 특성 이름을 일치시킴
    feature_names = list(X.columns)
    data = data.reindex(columns=feature_names, fill_value=0)

    # 특성 변수 스케일링
    data_scaled = scaler.transform(data)

    # 예측
    result = logreg.predict(data_scaled)
    if result[0] == 1:
        return '홈팀 승리'
    elif result[0] == 0.5:
        return '무승부'
    else:
        return '홈팀 패배'
```



최종 경기결과를 예측하는
예측 함수 구현

예측 함수 호출 예시

```
match_examples = [  
    ('Liverpool', 'Fulham'),  
    ('Manchester_United', 'Manchester_City'),  
    ('Barcelona', 'Real_Madrid'),  
    ('Bayern_Munich', 'Borussia_Dortmund'),  
    ('Paris_Saint_Germain', 'Marseille'),  
    ('Juventus', 'Inter_Milan'),  
    ('Chelsea', 'Arsenal'),  
    ('Tottenham_Hotspur', 'Leicester_City'),  
    ('Atletico_Madrid', 'Sevilla'),  
    ('Roma', 'Lazio')  
]  
  
for home_team, away_team in match_examples:  
    predicted_result = predict_match_result(home_team, away_team)  
    print(f"{home_team} vs {away_team}: {predicted_result}")
```

Liverpool vs Fulham: 홈팀 승리
Manchester_United vs Manchester_City: 홈팀 패배
Barcelona vs Real_Madrid: 홈팀 승리
Bayern_Munich vs Borussia_Dortmund: 홈팀 승리
Paris_Saint_Germain vs Marseille: 무승부
Juventus vs Inter_Milan: 홈팀 패배
Chelsea vs Arsenal: 홈팀 패배
Tottenham_Hotspur vs Leicester_City: 홈팀 승리
Atletico_Madrid vs Sevilla: 무승부
Roma vs Lazio: 홈팀 승리

예측 함수 결과확인



감 사 합 니 다